

PYTHON PROJECT PRESENTATION

Restaurant Billing System

A fast, accurate console workflow for menu selection, billing, tax calculation, and transaction history.

Python 3

JSON

Datetime



THE PROBLEM

Manual billing introduces friction at the exact moment service should feel effortless.

01

Calculation errors

Hand-added prices and taxes create avoidable inconsistencies in every checkout.

02

Lost history

Paper notes and ad hoc records make past orders difficult to retrieve or verify.

03

Slow service

Repeated arithmetic and rewriting bills increase customer waiting time.

PROJECT OVERVIEW

A lightweight console application turns a simple order into a reliable bill.

Design Intent

The system prioritizes **speed, accuracy**, and approachable file handling—without requiring databases or a graphical interface.

CLI-first

Small footprint

Beginner friendly

Automate billing

Convert selected menu items into a complete customer bill.

Apply tax reliably

Calculate a consistent 5% tax from the order subtotal.

Persist data simply

Use files that are readable, portable, and easy to inspect.

Teach foundations

Reinforce dictionaries, JSON, functions, loops, and file I/O.

WHO IT SERVES

Built for practical service environments—and practical learning.



Food stalls

Quickly total compact menus while keeping a lightweight record of each sale.



Quick-service eateries

Standardize checkout steps during busy, repeatable order cycles.



Python learners

Explore real-world use of loops, validation, JSON storage, and append-only logs.

A clear, file-based architecture keeps the system simple to run and easy to understand.



Command-line interface

Guided prompts collect the item choice, quantity, and order decisions.

Python 3 Input loops



Menu storage

menu_simple.json supplies item names and prices through JSON loading.

json.load()



Transaction log

order_log_simple.txt preserves timestamped receipt details in append mode.

datetime mode: a

CORE CAPABILITIES

Four building blocks deliver an end-to-end billing flow.

01 / MENU

Auto-initialized menu

Creates or loads a simple menu file so the application has a ready price catalog.



02 / ORDER

Interactive order taking

Prompts the user through item selection and validates entries before adding them.



03 / BILL

Subtotal + 5% tax

Separates subtotal, tax, and grand total for a transparent customer receipt.



04 / HISTORY

Timestamped logging

Writes completed transactions to a durable text record for later review.



Simple Python data structures keep the application readable and extensible.

Dictionaries

Map item names to prices for direct lookup during menu selection.

JSON parsing

`json.load()` reads structured menu data; `json.dump()` can initialize it.

Append-only history

`open(..., 'a')` adds each completed order without overwriting prior transactions.

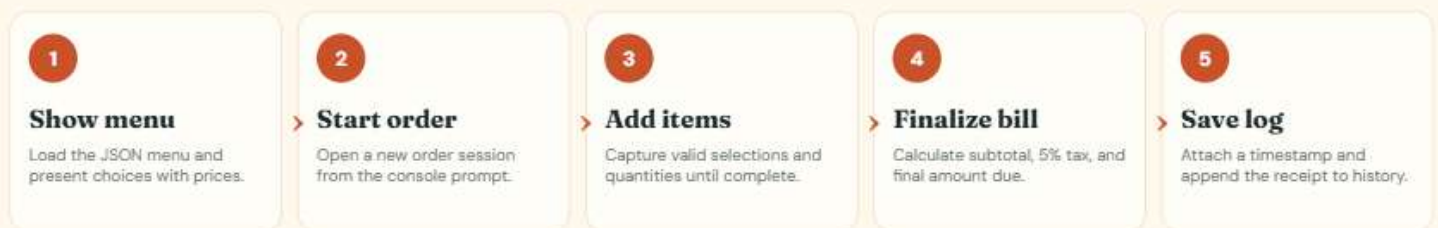
```
# menu data
menu = json.load(file)

# bill math
subtotal = sum(order.values())
tax = subtotal * 0.05
total = subtotal + tax

# durable log
with open('order_log_simple.txt', 'a') as log:
    log.write(receipt)
```

WORKFLOW

From menu display to saved transaction in five deliberate steps.



Result: the same order is visible to the customer and preserved for the operator.

ILLUSTRATIVE RECEIPT

A transparent receipt makes each charge—and the saved audit trail—easy to inspect.

RESTAURANT BILL	
Illustrative transaction	
Burger	\$8.50
Fries	\$3.00
Cola	\$2.50
Subtotal	\$14.00
Tax (5%)	\$0.70
TOTAL	\$14.70
Saved with timestamp	

[2026-09-29 12:30:45]

Order completed

Items: Burger, Fries, Cola

Subtotal: \$14.00

Tax (5%): \$0.70

Grand Total: \$14.70

→ appended to order_log_simple.txt

KEY TAKEAWAYS

A small project demonstrates production-relevant habits: reliable inputs, clear calculations, and persistent records.

What this system teaches

- ✓ Keep business logic explicit and traceable.
- ✓ Use structured files where a full database is unnecessary.
- ✓ Design console prompts to guide valid user behavior.
- ✓ Persist completed transactions for basic accountability.



GUI interface

Move from terminal prompts to touch-friendly checkout screens.



Database

Support search, reporting, multiple users, and richer inventory data.



Discount codes

Add promotions, coupons, and conditional pricing rules.



Sales analytics

Summarize best sellers, taxes, and performance over time.

CONCLUSION

Simple files. **Reliable bills.** **Faster service.**

The Restaurant Billing System converts core Python concepts into a practical, extensible operational tool.

Q&A